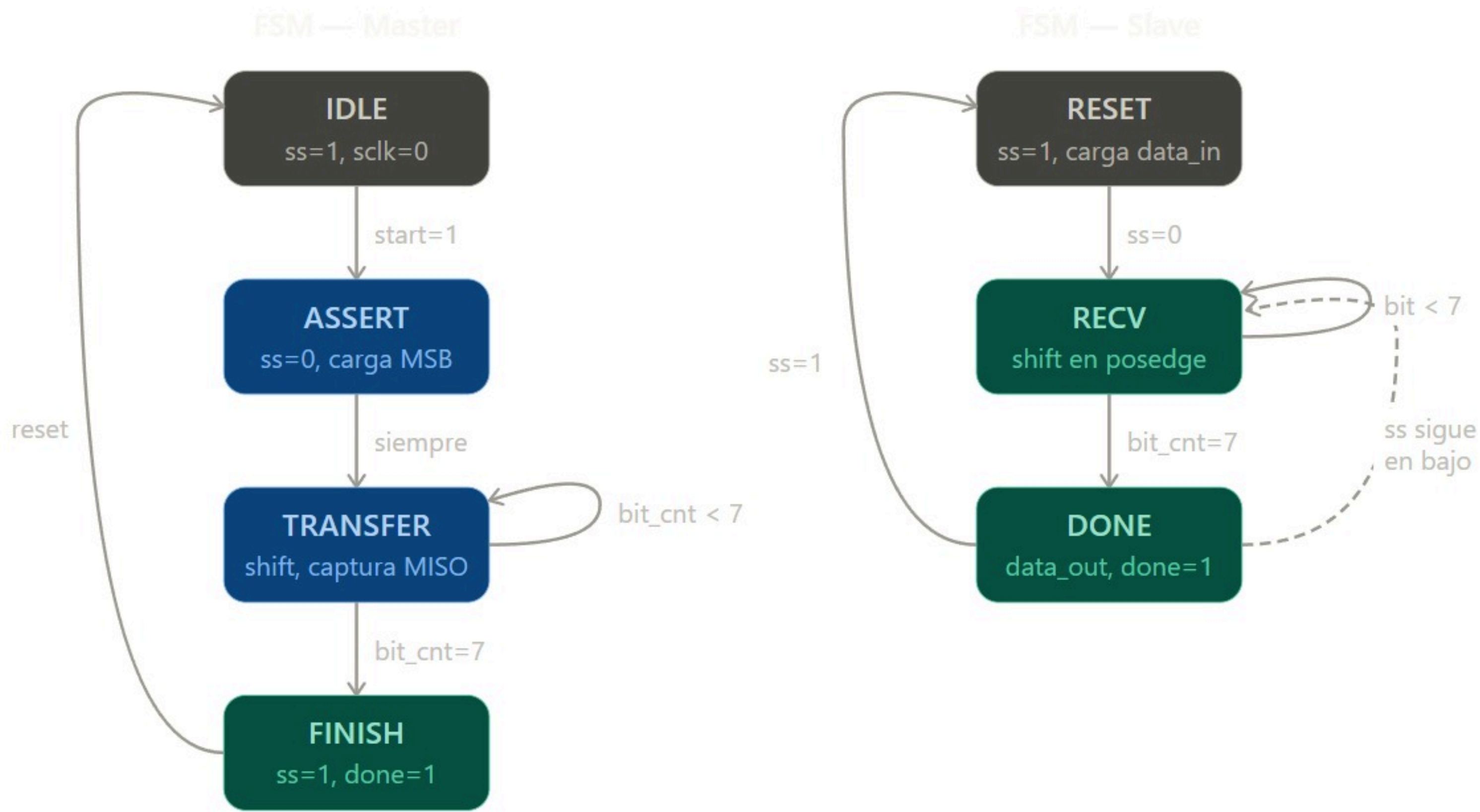




Formato de trama — 1 byte (8 bits)



UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

INGENIERÍA EN COMPUTACIÓN

ARQUITECTURA DE COMPUTADORAS

2.Avance Reporte

Implementación de Módulo SPI

Nombres:

- Luis Giovanni Rodríguez Hernandez
- Sara Iriel Alonso Jaquez
- Leonel Octavio Rios Loreto
- Jorge Abraham Moran Encina

Fecha: 13 de Marzo de 2026

Implementación de Módulo SPI

1. Introducción

En los sistemas digitales es común que un microcontrolador, procesador o FPGA necesite comunicarse con distintos dispositivos periféricos para intercambiar información. Para lograr esta comunicación se utilizan diversos protocolos de comunicación serial, los cuales permiten la transmisión de datos entre circuitos integrados de manera organizada y eficiente.

Uno de los protocolos más utilizados en sistemas embebidos es el SPI (Serial Peripheral Interface), un protocolo de comunicación serial síncrono que permite la transferencia de datos entre un dispositivo maestro y uno o varios dispositivos esclavos. Este protocolo se caracteriza por su simplicidad de implementación, alta velocidad de transmisión y capacidad de comunicación full-duplex, lo que lo hace ideal para aplicaciones donde se requiere una comunicación rápida entre dispositivos electrónicos.

El protocolo SPI se utiliza combinado para la comunicación con dispositivos como memorias, sensores, convertidores analógico-digitales y pantallas, y es ampliamente implementado en microcontroladores y FPGAs. En este trabajo se presenta el análisis del protocolo SPI, así como la explicación de sus modos de operación basados en la polaridad y fase del reloj, los cuales determinan la forma en que los datos son transmitidos y sincronizados durante la comunicación.

2. Marco Teórico

- Protocolo SPI (Interfaz periférica en serie)

El Serial Peripheral Interface (SPI) es un protocolo de comunicación síncrona serial utilizado para intercambiar datos entre dispositivos electrónicos a corta distancia dentro de un mismo sistema. En este tipo de comunicación, los datos se transmiten bit a bit de forma serial, sincronizados mediante una señal de reloj generada por el dispositivo maestro.

El funcionamiento del protocolo se basa en una estructura maestro-esclavo, donde el maestro controla el bus de comunicación y uno o más dispositivos esclavos responden a sus comandos. Para que se produzca la comunicación, el maestro activa la línea de selección correspondiente al dispositivo esclavo con el cual desea comunicarse. Una vez seleccionado el esclavo, comienza la transmisión de datos sincronizada por la señal del reloj.

Una característica importante del protocolo SPI es que permite comunicación full-duplex, es decir, los datos pueden enviarse y recibirse simultáneamente entre el maestro y el esclavo. Esto se logra mediante registros de desplazamiento internos en los dispositivos, que permiten enviar y recibir bits al mismo tiempo durante cada ciclo de reloj.

Se utilizan generalmente cuatro líneas principales de comunicación:

1. SCLK (Serial Clock)

Es la señal de reloj generada por el dispositivo maestro. Esta señal sincroniza la transmisión de datos entre los dispositivos conectados al bus.

2. MOSI (Master Out Slave In)

Es la línea por la cual el dispositivo maestro envía datos hacia el dispositivo esclavo.

3. MISO (Master In Slave Out)

Es la línea utilizada por el dispositivo esclavo para enviar datos de regreso al maestro.

4. SS o CS (Slave Select / Chip Select)

Es una señal de control que permite seleccionar qué dispositivo esclavo participará en la comunicación. Cuando esta línea se activa, el esclavo correspondiente queda habilitado para recibir y transmitir datos.

En sistemas donde existen múltiples dispositivos esclavos, cada uno suele tener su propia línea SS , mientras que las líneas MOSI, MISO y SCLK son compartidas por todos los dispositivos conectados al bus.

- Modos de operación del SPI (Clock Polarity y Clock Phase)

El protocolo SPI permite configurar la forma en que los datos son sincronizados con la señal del reloj. Esta configuración se realiza mediante dos parámetros fundamentales:

1. CPOL (Polaridad del reloj)
2. CPHA (Fase del reloj)

Estos dos parámetros determinan el comportamiento del reloj y el momento exacto en el que los datos son enviados y capturados durante la comunicación.

- Polaridad del reloj (CPOL)

La polaridad del reloj (CPOL) define el nivel lógico que tiene la señal de reloj cuando el bus está en estado inactivo, es decir, cuando no se está transmitiendo información.

Existen dos configuraciones posibles:

CPOL = 0

El reloj permanece en nivel bajo cuando el autobús está inactivo.

CPOL = 1

El reloj permanece en nivel alto cuando el autobús está inactivo.

Esta configuración determina cuál será el primer flanco del reloj que ocurrirá cuando comience la transmisión de datos.

- Fase del reloj (CPHA)

La fase del reloj (CPHA) determina en qué flanco del reloj se leen o se capturan los datos durante la comunicación.

Las dos configuraciones posibles son:

CPHA = 0

Los datos se capturan en el primer flanco del reloj y se cambian en el segundo.

CPHA = 1

Los datos se capturan en el segundo flanco del reloj .

Esto permite ajustar la sincronización de los datos dependiendo de las características de los dispositivos conectados al autobús.

- Modos SPI

La combinación de CPOL y CPHA genera cuatro modos de operación diferentes en el protocolo SPI.

Modo SPI	CPOL	CPHA
Modo 0	0	0
Modo 1	0	1
Modo 2	1	0
Modo 3	1	1

Cada modo define en qué flanco del reloj se transmite y se capturan los datos, por lo que el maestro y el esclavo deben configurarse en el mismo modo para que la comunicación funcione correctamente.

Referencias

CoLtd, C. E. E. T. (2023, 4 JULIO). EXPLICACIÓN DETALLADA DEL PROTOCOLO DE COMUNICACIÓN SPI_DINÁMICA DE LA INDUSTRIA_BLOG_CHENGDU EBYTE ELECTRONIC TECHNOLOGY Co.,LTD-ES. CHENGDU EBYTE ELECTRONIC TECHNOLOGY Co.,LTD. <https://www.es-ebYTE.COM/news/470>

¿QUÉ ES UNA INTERFAZ SPI (SERIAL PERIPHERAL INTERFACE)? | DEFINICIÓN. (S. F.). DIGI. <https://es.digi.com/resources/definitions/spi>

SPI - ARDUWIKI. (S. F.). <https://arduwiki.perut.org/index.php/SPI>

DHAKER, P. (2018). INTRODUCTION TO SPI INTERFACE. *ANALOG DIALOGUE*, 52(3), 49-53.

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

INGENIERÍA EN COMPUTACIÓN

ARQUITECTURA DE COMPUTADORAS

3.Avance Video

Implementación de Módulo SPI (Guión del Video)

Nombres:

- Luis Giovanni Rodríguez Hernandez
- Sara Iriel Alonso Jaquez
- Leonel Octavio Rios Loreto
- Jorge Abraham Moran Encima

Fecha: 13 de Marzo de 2026

Guion Video SPI

Hola, en este video se explicará qué es el protocolo SPI, cómo funciona y dónde se utiliza.

El **SPI**, que significa *Serial Peripheral Interface*, es un protocolo de comunicación serial que se usa para conectar microcontroladores con diferentes dispositivos periféricos. Fue desarrollado por Motorola y es muy usado en sistemas embebidos.

Este protocolo se utiliza en muchos dispositivos electrónicos como **memorias flash, tarjetas SD, sensores, pantallas LCD y convertidores analógico-digitales**. Una de sus ventajas es que permite transmitir datos de forma rápida y sencilla entre un dispositivo maestro y uno o varios dispositivos esclavos.

El funcionamiento de SPI se basa en **cuatro líneas principales de comunicación**.

La primera es **SCLK**, que es la señal de reloj generada por el maestro y sincroniza la transmisión de datos.

La segunda es **MOSI**, que significa *Master Out Slave In* y se utiliza para enviar datos del maestro al esclavo.

La tercera es **MISO**, *Master In Slave Out*, que envía datos del esclavo al maestro.

Y la cuarta es **SS o CS**, llamada *Slave Select*, que permite al maestro seleccionar con qué dispositivo esclavo quiere comunicarse.

Durante la comunicación, los datos se transmiten **bit por bit sincronizados con el reloj**. Mientras el maestro envía datos por MOSI, el esclavo puede responder al mismo tiempo por MISO.

En una implementación digital, como con Verilog, el controlador SPI normalmente utiliza una **máquina de estados finitos** con estados mínimos como:

IDLE, donde el sistema espera iniciar la comunicación;

SETUP, donde se prepara la transmisión;

TRANSMIT, donde se envían los datos;

y **DONE**, donde finaliza la operación.

Finalmente, la **trama de SPI** normalmente está formada por un conjunto de bits, generalmente **8 bits**, que representan el dato que se envía entre los dispositivos.

En resumen, SPI es un protocolo simple, rápido y muy utilizado para la comunicación entre dispositivos digitales en sistemas electrónicos.

Story board

Escena 1 – Introducción

Narración: Presentación del tema y explicación breve de qué es SPI.

Visual: Título en pantalla “*Protocolo SPI*” y un diagrama simple de un microcontrolador conectado a otros dispositivos.

Escena 2 – Qué es SPI y dónde se usa

Narración: Explicación de qué significa SPI y algunos dispositivos donde se utiliza.

Visual: Imágenes o iconos de dispositivos como memoria flash, tarjeta SD, sensores o pantallas LCD.

Escena 3 – Líneas de comunicación

Narración: Explicación de las cuatro líneas principales: SCLK, MOSI, MISO y SS.

Visual: Diagrama simple que muestre un dispositivo maestro y un esclavo conectados por las cuatro líneas.

Escena 4 – Cómo funciona la comunicación

Narración: Explicación de que los datos se transmiten **bit por bit sincronizados con el reloj**.

Visual: Animación o imagen mostrando bits moviéndose entre el maestro y el esclavo.

Escena 5 – Máquina de estados y conclusión

Narración: Mención de los estados básicos del controlador SPI (IDLE, SETUP, TRANSMIT y DONE) y cierre del video.

Visual: Diagrama simple de estados y texto final resumiendo que SPI es un protocolo rápido y sencillo para comunicación entre dispositivos.

GUION POR ESCENAS:

Escena 1 – Introducción

Hola, en este video se explicará qué es el protocolo SPI, cómo funciona y dónde se utiliza.

Escena 2 – Qué es SPI y dónde se usa

El SPI, que significa Serial Peripheral Interface, es un protocolo de comunicación serial que se usa para conectar microcontroladores con diferentes dispositivos periféricos. Fue desarrollado por Motorola y es muy usado en sistemas embebidos.

Este protocolo se utiliza en muchos dispositivos electrónicos como memorias flash, tarjetas SD, sensores, pantallas LCD y convertidores analógico-digitales. Una de sus ventajas es que permite transmitir datos de forma rápida y sencilla entre un dispositivo maestro y uno o varios dispositivos esclavos.

Escena 3 – Líneas de comunicación

El funcionamiento de SPI se basa en cuatro líneas principales de comunicación.

La primera es SCLK, que es la señal de reloj generada por el maestro y sincroniza la transmisión de datos.

La segunda es MOSI, que significa *Master Out Slave In* y se utiliza para enviar datos del maestro al esclavo.

La tercera es MISO, *Master In Slave Out*, que envía datos del esclavo al maestro.

Y la cuarta es SS o CS, llamada *Slave Select*, que permite al maestro seleccionar con qué dispositivo esclavo quiere comunicarse.

Escena 4 – Cómo funciona la comunicación

Durante la comunicación, los datos se transmiten bit por bit sincronizados con el reloj. Mientras el maestro envía datos por MOSI, el esclavo puede responder al mismo tiempo por MISO.

Escena 5 – Máquina de estados y conclusión

En una implementación digital, como con Verilog, el controlador SPI normalmente utiliza una máquina de estados finitos con estados mínimos como:

- IDLE, donde el sistema espera iniciar la comunicación
- SETUP, donde se prepara la transmisión
- TRANSMIT, donde se envían los datos
- DONE, donde finaliza la operación

Finalmente, la trama de SPI normalmente está formada por un conjunto de bits, generalmente 8 bits, que representan el dato que se envía entre los dispositivos.

En resumen, SPI es un protocolo simple, rápido y muy utilizado para la comunicación entre dispositivos digitales en sistemas electrónicos.

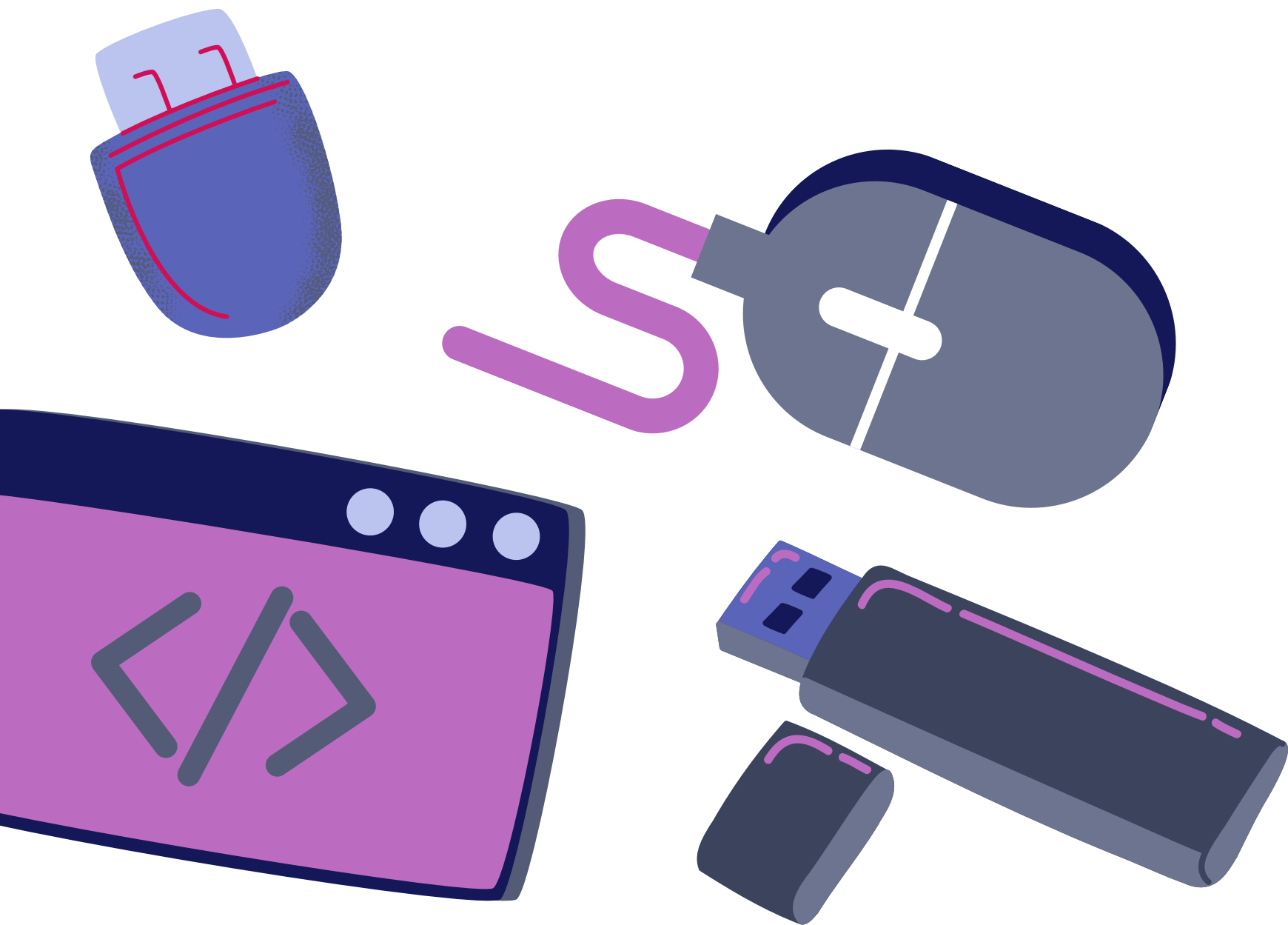


Arquitectura de Computadoras

MÓDULO SPI

Implementación

ÍNDICE



01. Introducción

02. Usos

03. Funcionamiento de SPI

04. TSCLK

05. MOSI y MISO

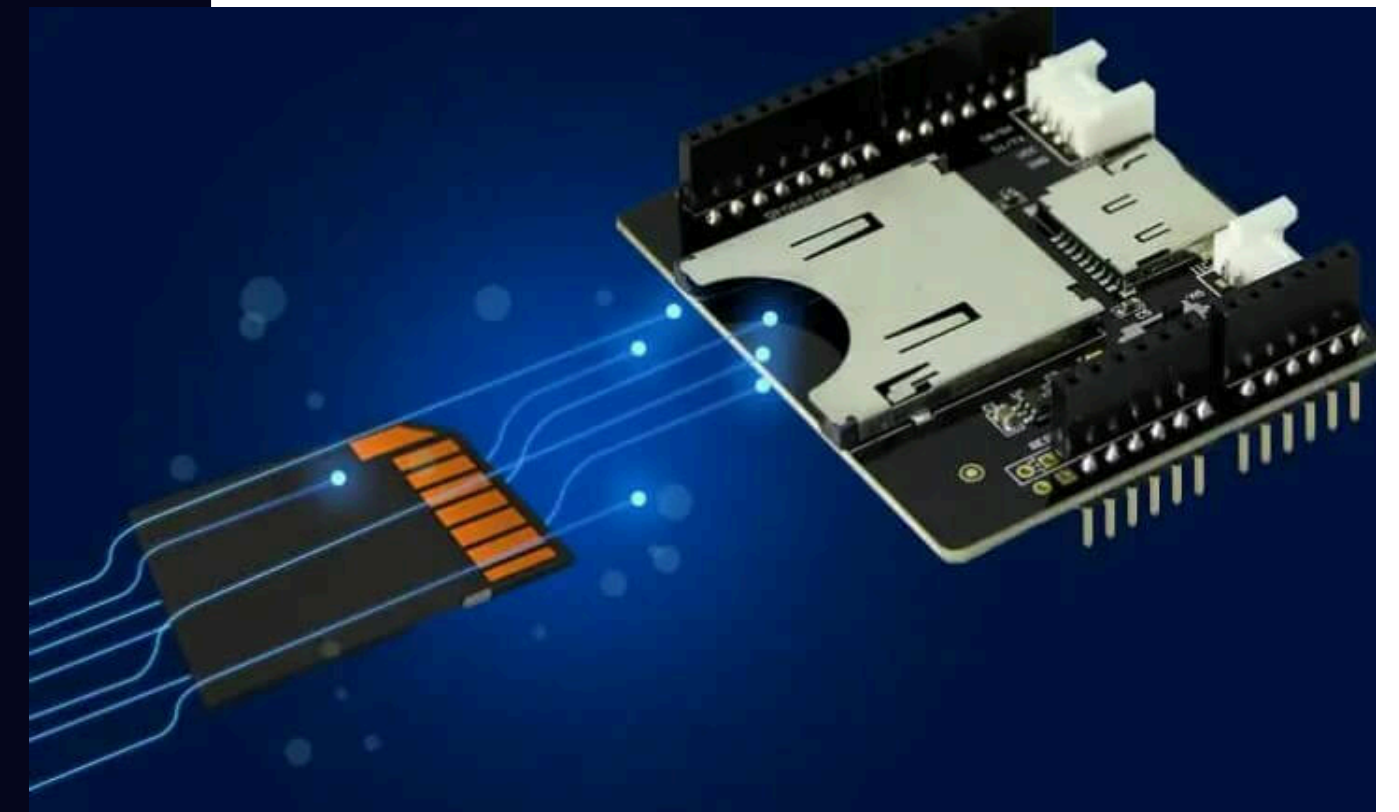
06. SS o CS

07. Verilog

08. Techbench

1.Introducción

El SPI (Serial Peripheral Interface) es un protocolo de comunicación síncrono serial utilizado para intercambiar datos entre un dispositivo maestro y uno o más dispositivos esclavos . Se caracteriza por ser rápido y sencillo, ya que utiliza cuatro líneas principales: MOSI, MISO, SCLK y SS.



2.Usos

Este protocolo se usa comúnmente para comunicar dispositivos como memorias, sensores, pantallas y otros periféricos .

En el diseño digital, como en Verilog , se implementa un módulo SPI para controlar esta comunicación dentro de sistemas en FPGA y otros.

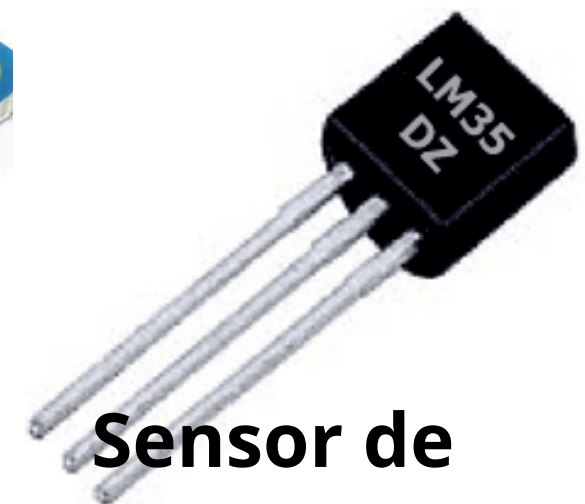
Memorias Flash



Pantallas LCD



Sensores Acelerometro y giroscopio



Sensor de temperatura

Formato de trama

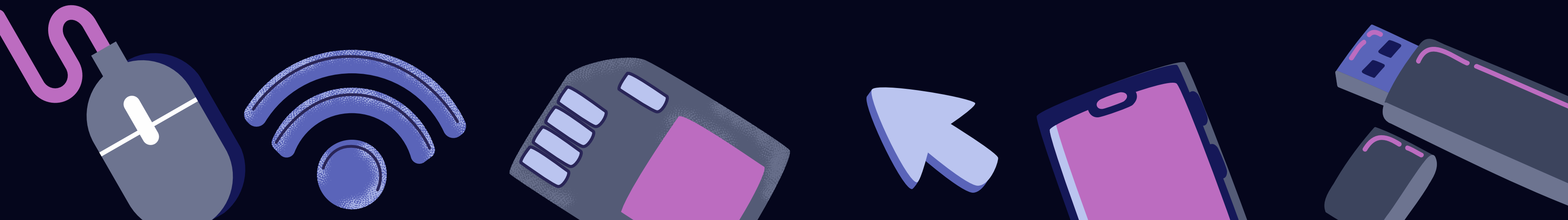
El bus tiene 4 señales. SS es el que inicia y termina la comunicación — cuando baja, el slave despierta; cuando sube, vuelve a reset. SCLK es el reloj que genera el master. MOSI lleva datos del master al slave, MISO los regresa del slave al master.

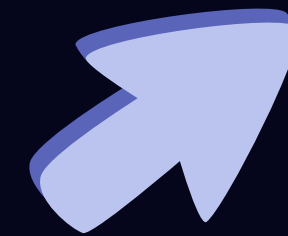
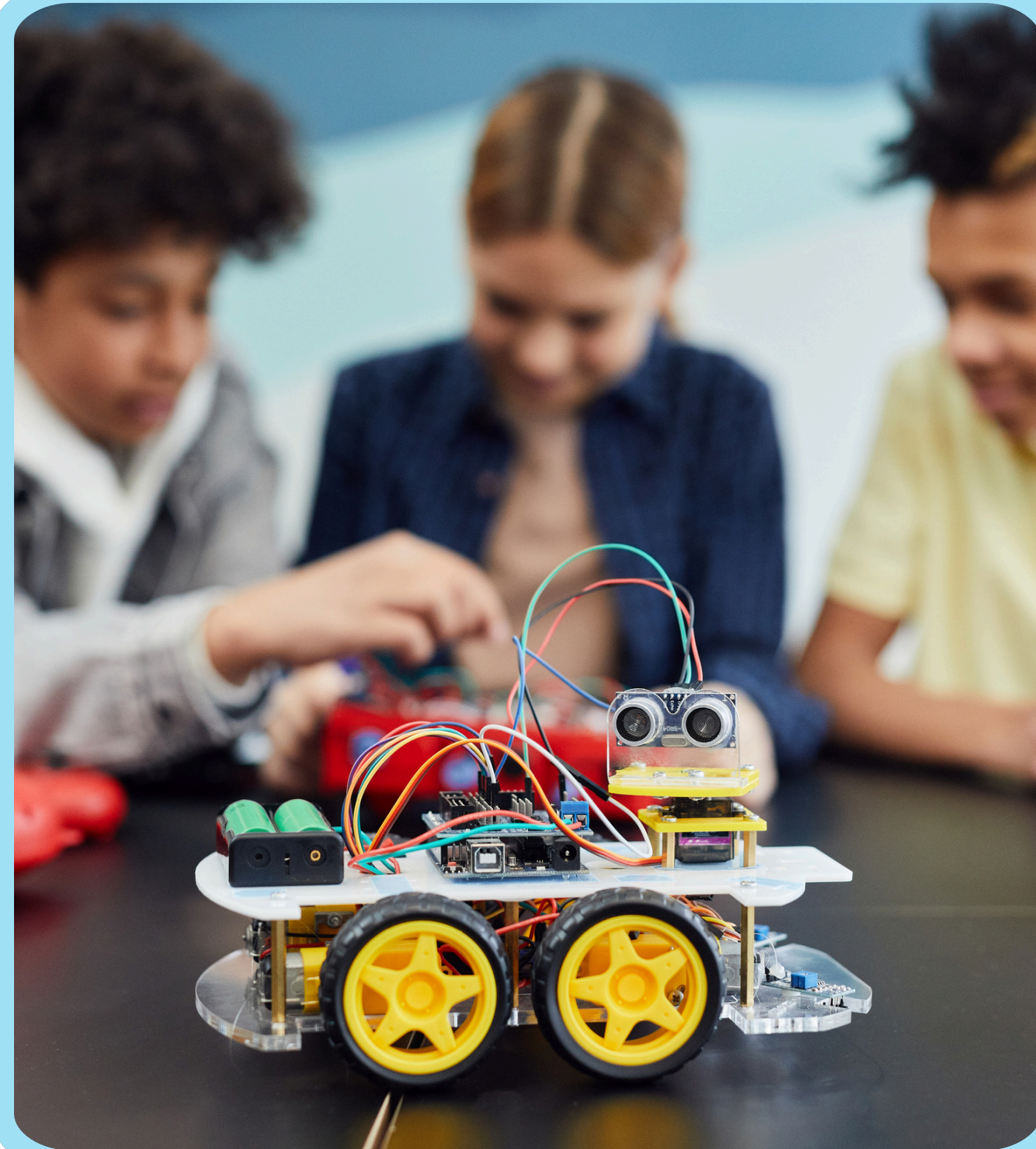
El bit más significativo siempre va primero — MSB first — y done pulsa exactamente 1 ciclo cuando termina la transferencia.





3.FUNCIONAMIENTO DE SPI





Slave

El slave es el módulo más sencillo. Tiene solo dos estados reales: RESET y RECV. Mientras SS está en alto, el slave está en reset — carga el dato que quiere enviar en shift_reg y pone el contador en cero. En cuanto SS baja, entra a RECV y en cada flanco de subida de SCLK desplaza el registro: mete el bit de MOSI por la derecha y saca el MSB por MISO automáticamente, porque miso es un simple assign al bit 7 del registro.



Hay cuatro decisiones de diseño ha destacar:

1ero

Miso es wire y no reg — no necesito un always, un assign directo es suficiente y más eficiente.

2do

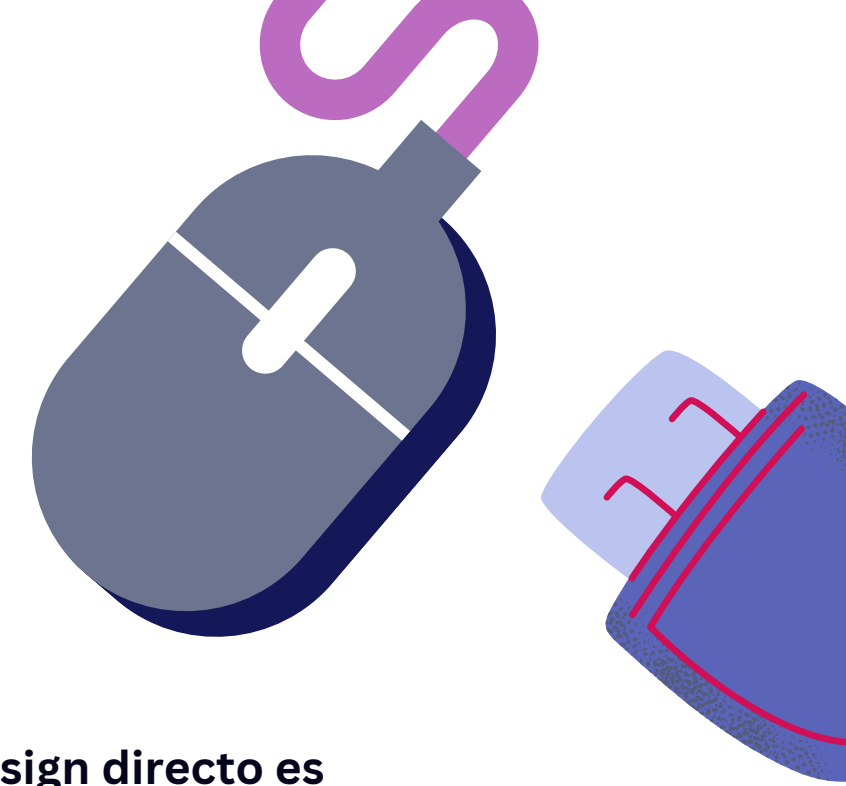
El reset es asíncrono con SS, lo que permite recargar data_in entre tramas sin necesitar un reset global.

3ro

El shift y la captura ocurren en un solo posedge sclk, minimizando la latencia.

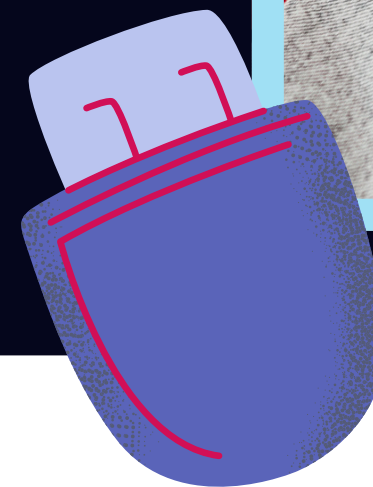
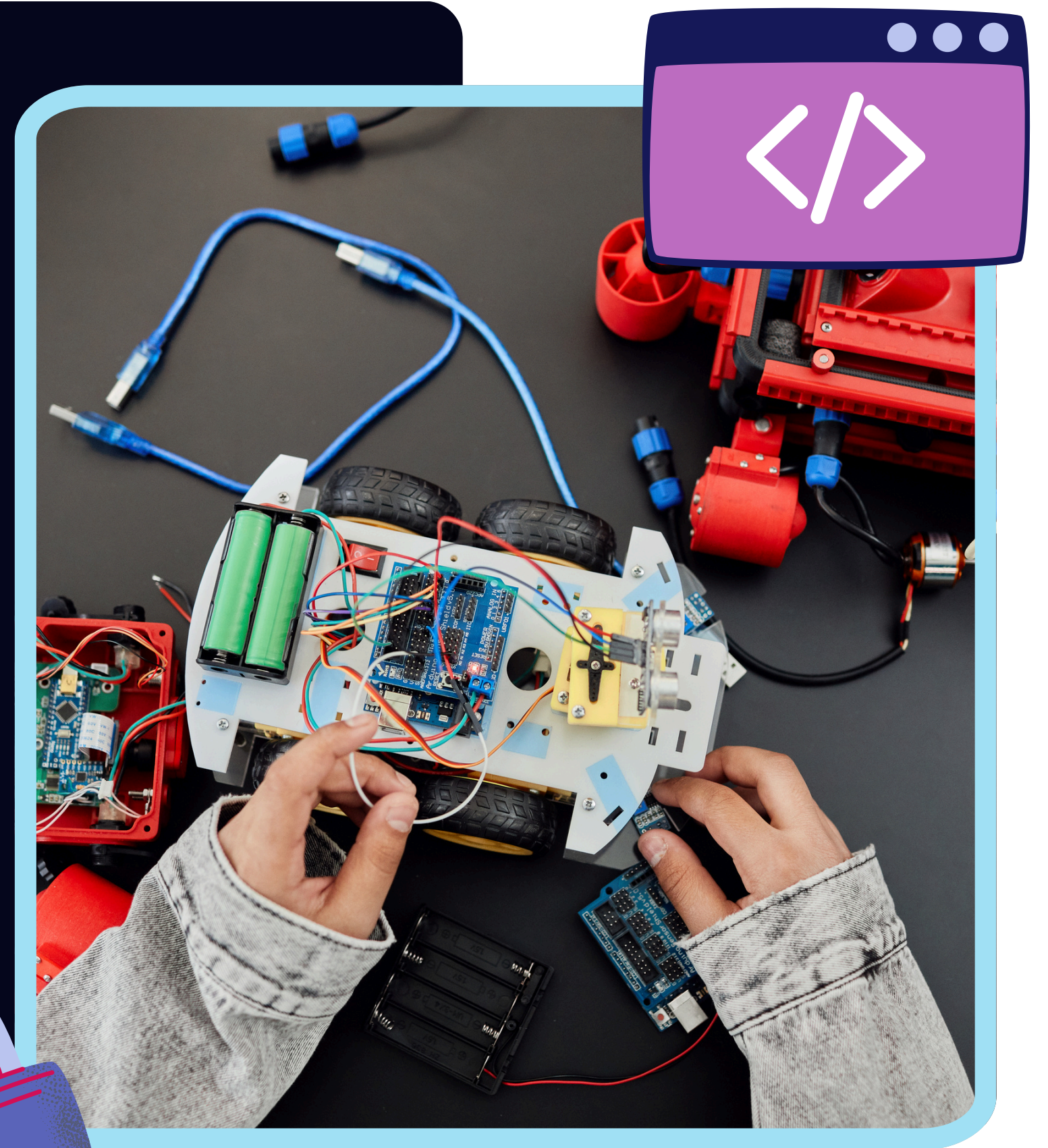
4to

Done sí es reg porque necesita almacenar ese pulso de 1 ciclo hasta el siguiente flanco de reloj.



Master

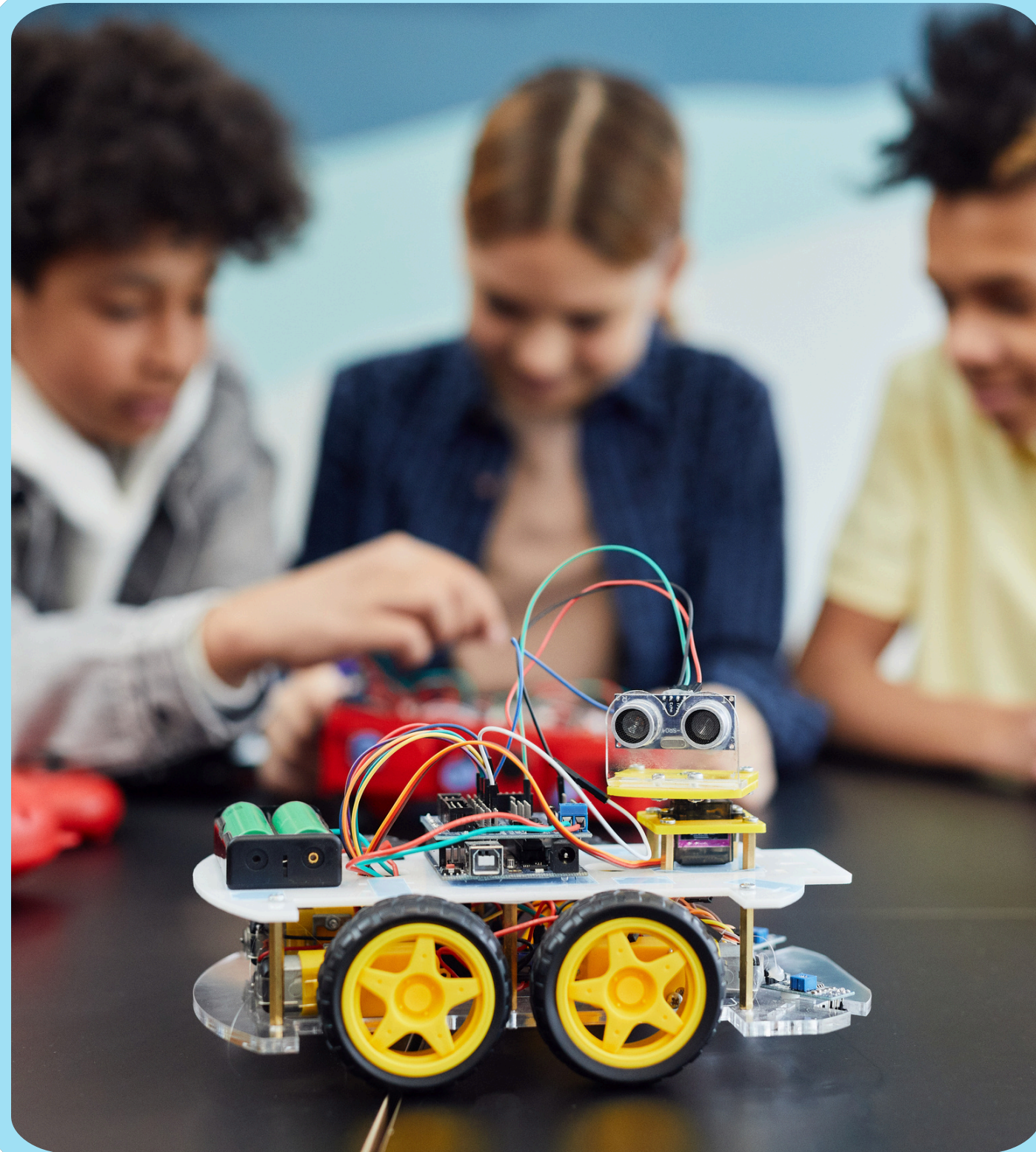
El master tiene cuatro estados. IDLE espera la señal start. ASSERT baja SS y precarga el MSB en MOSI — este estado es necesario porque si mandamos el primer flanco de SCLK antes de bajar SS, el slave no ha salido de reset y pierde el primer bit. TRANSFER es donde ocurre toda la transferencia: en cada semiciclo bajo de SCLK actualizo MOSI con el siguiente bit y capturo MISO en capture_reg. Cuando bit_cnt llega a 7, paso a FINISH, subo SS y pulso done.





EL PARÁMETRO CLK_DIV ME PERMITE CONTROLAR LA VELOCIDAD DE SCLK SIN CAMBIAR LA LÓGICA:

Con `CLK_DIV=1` el SCLK es la mitad del reloj del sistema, con `CLK_DIV=5` es un décimo. Y la señal busy bloquea nuevos start mientras hay una transferencia activa, evitando colisiones.



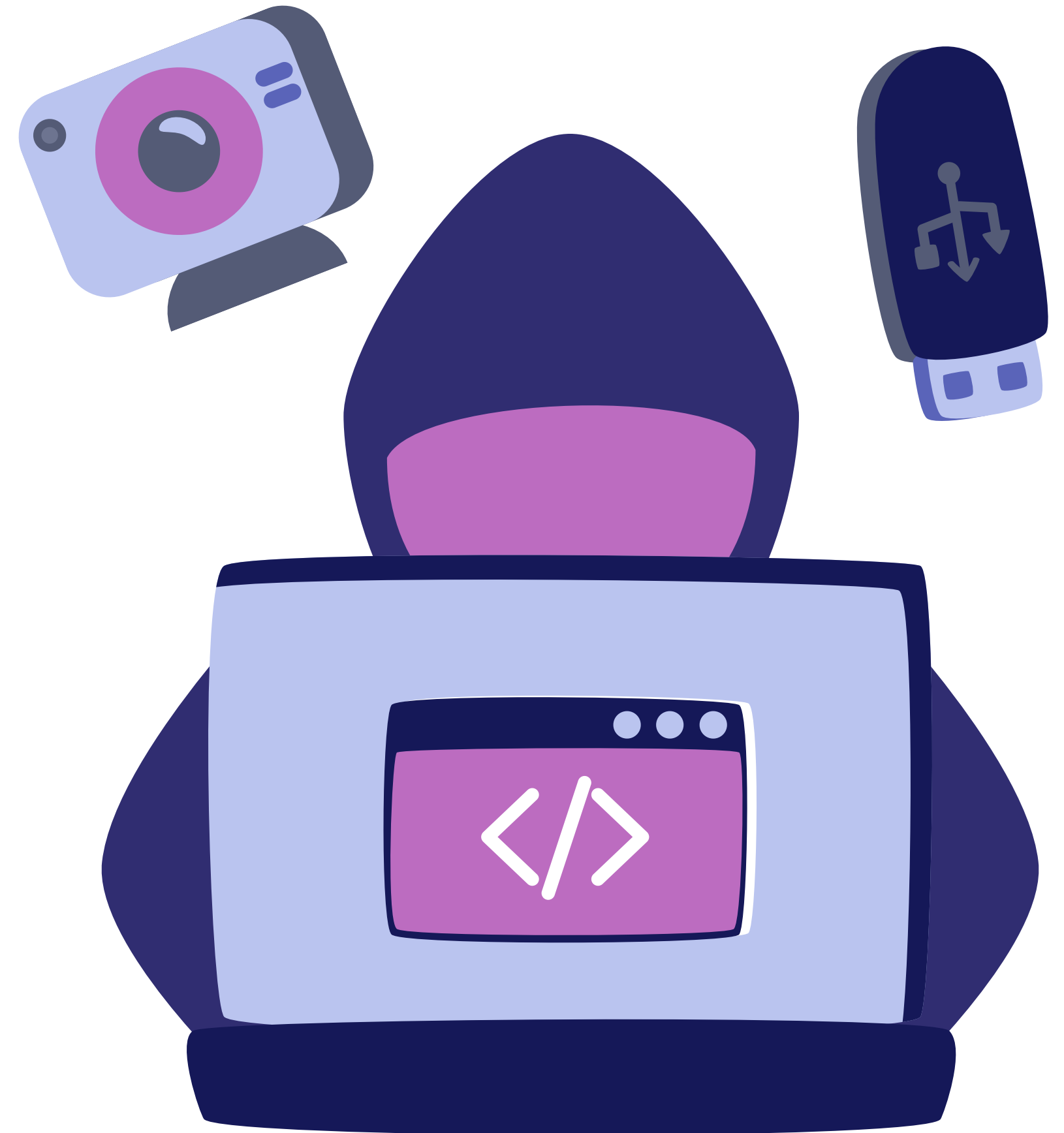
Testbench

El testbench instancia un master y un slave y los conecta directamente por el bus SPI — sclk, ss, mosi y miso son los mismos wires para ambos.

Definí una tarea `send_byte(master_data, slave_data)` que recarga el slave con su dato, pulsa start en el master, y espera el flanco de done para imprimir los resultados. Esto me permite escribir casos de prueba en una sola línea.

Se utilizaron cuatro patrones:
0xA5 y 0xFF/0x00 para verificar valores extremos,
0x55/0xAA que es el patrón checker-board para
detectar si algún bit se invierte, y 0x12/0x34 como
valores arbitrarios generales.

Para simular se compila con iverilog, se ejecuta con vvp,
y se puede visualizar con GTKWave.



4 pares independientes

Para escalar a 4 pares se utilizó generate/genvar, que en Verilog permite instanciar módulos en un loop sin repetir código.

Cada iteración crea un master y un slave con su propio bus privado — sus propios sclk, ss, mosi y miso. No hay ninguna señal compartida entre pares, lo que elimina cualquier posibilidad de colisión o interferencia.

El costo es en pines — necesitamos 4 buses completos — pero la simplicidad y la independencia lo justifican.

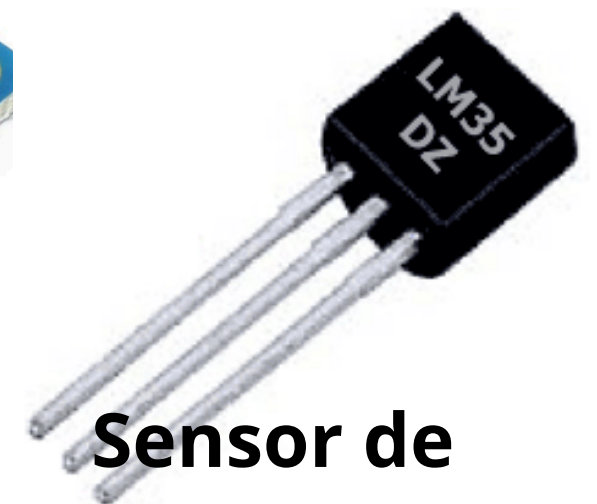
Memorias Flash



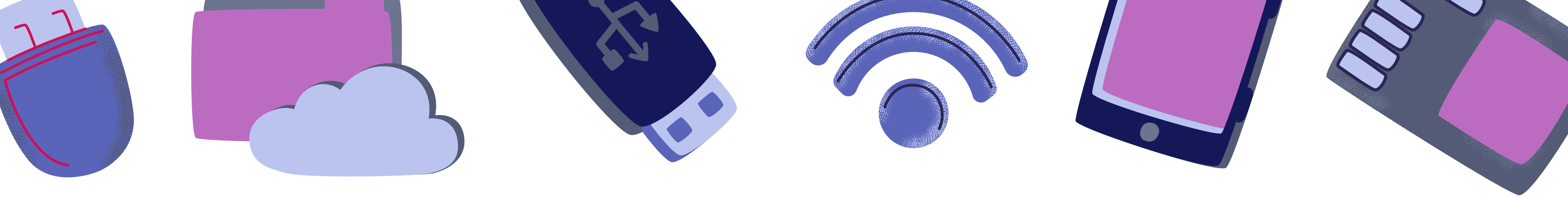
Pantallas LCD



Sensores Acelerometro y giroscopio



Sensor de temperatura



CONCLUSION

Para cerrar: el slave quedó con 2 estados mínimos gracias al reset asíncrono con SS. El master necesita 4 estados, siendo ASSERT el más importante para garantizar que el slave esté listo antes del primer bit.

La transferencia es full-duplex de forma natural — no hay lógica extra para eso, solo ocurre por la estructura del shift register. El testbench verifica todo con una tarea reutilizable y 4 patrones bien elegidos. Y el escalado a 4 pares no requirió modificar los módulos originales, solo instanciarlos con generate.

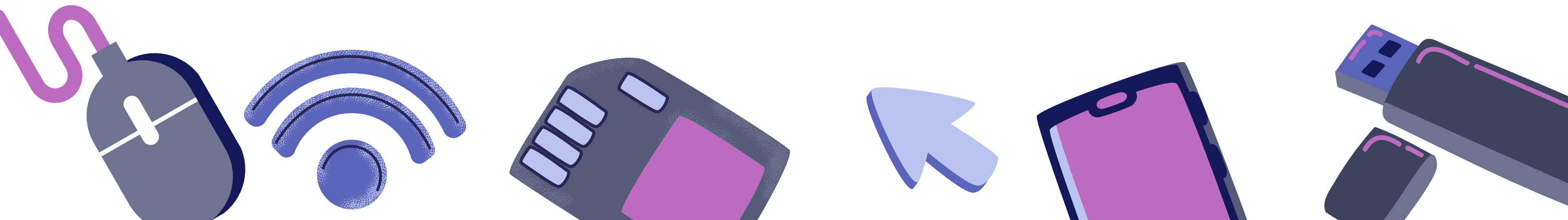
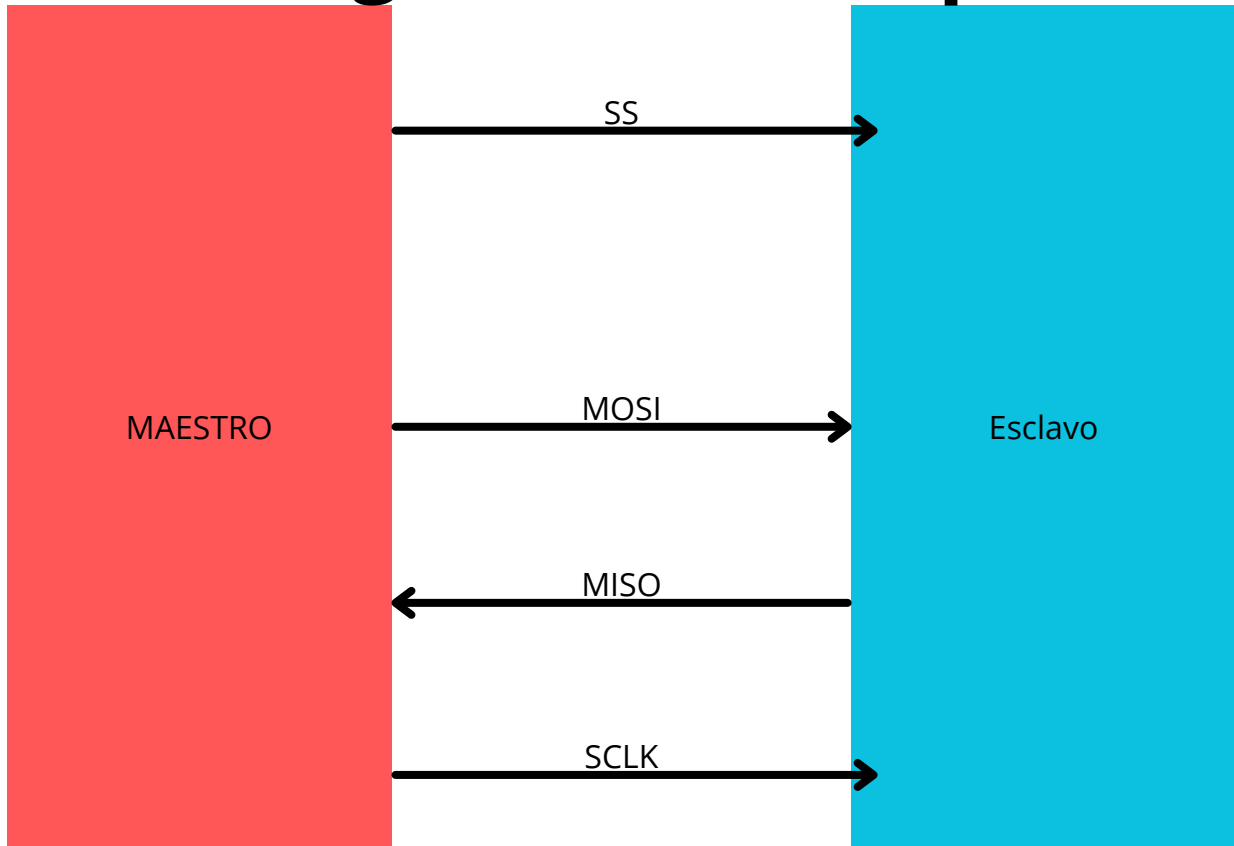


Diagrama de Bloques



ROLES Y RESPONSABILIDADES:

Luis Giovanni Rodríguez Hernandez:

-Guión Literario/Story board y Presentación

Sara Iriel Alonso Jaquez:

-Presentación y Reporte

Leonel Octavio Rios Loreto:

-Código, techbench

Jorge Abraham Moran Encina:

-Código, techbench